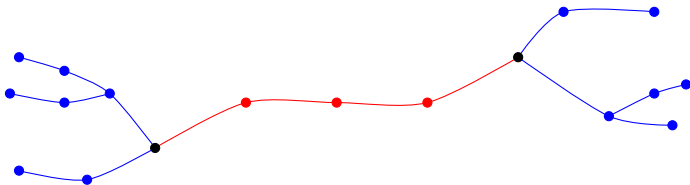


Landmark Hub Labeling: Improved Bounds and Faster Query Answering

Justine Cauvi, Ruoying Li, Sabine Storandt

École Normale Supérieure de Lyon, France
University of Konstanz, Germany

ATMOS 2024



Plan

1 Introduction

2 Results

- Label size gaps
- Experimental label size comparison
- Faster query answering

3 Conclusion

Motivation

Problem : Determining the **shortest path distance** between a given pair of nodes in a **weighted graph** $G(V, E, c)$

Applications :

- **Road network** : navigation systems, urban planning, vehicle routing
- VLSI design
- Data package routing in computer networks
- Social network analysis

Motivation

Problem : Determining the **shortest path distance** (and an associated path) between a given pair of nodes in a **weighted graph** $G(V, E, c)$

Naive solution : Dijkstra's algorithm : $\mathcal{O}(m + n \log n)$

Problem : **Many queries** issued on the **same graph**

Goal : **Preprocessing-based** techniques

- **Faster query answering** than Dijkstra
- **Subquadratic space** consumption

Motivation

Problem : Determining the **shortest path distance** (and an associated path) between a given pair of nodes in a **weighted graph** $G(V, E, c)$

Naive solution : Dijkstra's algorithm : $\mathcal{O}(m + n \log n)$

Problem : **Many queries** issued on the **same graph**

Goal : **Preprocessing-based** techniques

- **Faster query answering** than Dijkstra
- **Subquadratic space** consumption

Hub Labeling (HL)

Assumptions and notations :

- Undirected graph $G = (V, E, c)$ with positive edge costs $c : E \rightarrow \mathbb{R}^+$
- Unique shortest path: $\pi(s, t)$, cost: $c(\pi(s, t))$

Hub Labeling (HL) : [Cohen et al. 2003]

- Labeling: $L : V \rightarrow 2^V$
- Hub cover property: $\forall s, t \in V, \exists v \in L(s) \cap L(t), v \in \pi(s, t)$



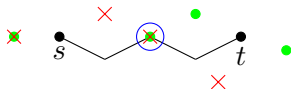
Hub Labeling (HL)

Assumptions and notations :

- Undirected graph $G = (V, E, c)$ with positive edge costs $c : E \rightarrow \mathbb{R}^+$
- Unique shortest path: $\pi(s, t)$, cost: $c(\pi(s, t))$

Hub Labeling (HL) : [Cohen et al. 2003]

- Labeling: $L : V \rightarrow 2^V$
- Hub cover property: $\forall s, t \in V, \exists v \in L(s) \cap L(t), v \in \pi(s, t)$



HL : Query answering

Distance query answering :

- Store distance $c(\pi(v, w))$ for each label $w \in L(v)$
- $c(\pi(s, t)) = \min_{v \in L(s) \cap L(t)} (c(\pi(s, v)) + c(\pi(v, t)))$
- Store label sets sorted by node ID
- Perform merge-like procedure to find common IDs

$$\implies \mathcal{O}(|L(s)| + |L(t)|)$$

Space consumption : $\mathcal{O}(\sum_{v \in V} |L(v)|)$

Hence, the maximum individual label set size as well as the total labeling size should be **as small as possible**.

HL : Query answering

Distance query answering :

- Store distance $c(\pi(v, w))$ for each label $w \in L(v)$
- $c(\pi(s, t)) = \min_{v \in L(s) \cap L(t)} (c(\pi(s, v)) + c(\pi(v, t)))$
- Store label sets sorted by node ID
- Perform merge-like procedure to find common IDs

$$\implies \mathcal{O}(|L(s)| + |L(t)|)$$

Space consumption : $\mathcal{O}(\sum_{v \in V} |L(v)|)$

Hence, the maximum individual label set size as well as the total labeling size should be **as small as possible**.

HL : Query answering

Distance query answering :

- Store distance $c(\pi(v, w))$ for each label $w \in L(v)$
- $c(\pi(s, t)) = \min_{v \in L(s) \cap L(t)} (c(\pi(s, v)) + c(\pi(v, t)))$
- Store label sets sorted by node ID
- Perform merge-like procedure to find common IDs

$$\implies \mathcal{O}(|L(s)| + |L(t)|)$$

Space consumption : $\mathcal{O}(\sum_{v \in V} |L(v)|)$

Hence, the maximum individual label set size as well as the total labeling size should be **as small as possible**.

Landmark Hub Labeling (LHL)

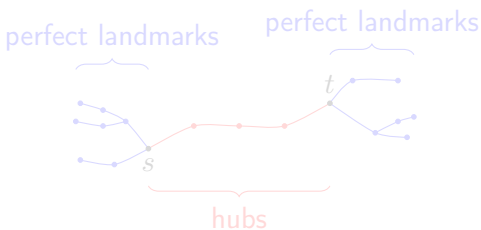
Landmark Hub Labeling (LHL) : [Storandt 2022]

- Generalisation of HL
- Allows also perfect landmark to cover a pair of nodes

Definition

A node $\ell \in V$ is a perfect landmark for $s, t \in V$ if :

$|c(\pi(s, \ell)) - c(\pi(t, \ell))| = c(\pi(s, t)) \Leftrightarrow t$ lies on a shortest path from s to ℓ or s lies on a shortest path from ℓ to t



Landmark Hub Labeling (LHL)

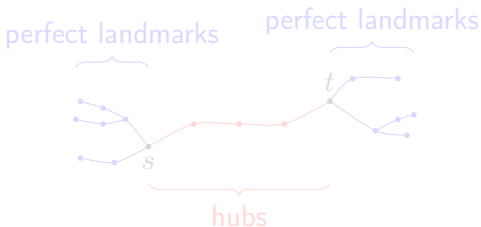
Landmark Hub Labeling (LHL) : [Storandt 2022]

- Generalisation of HL
- Allows also perfect landmark to cover a pair of nodes

Definition

A node $\ell \in V$ is a perfect landmark for $s, t \in V$ if :

$|c(\pi(s, \ell)) - c(\pi(t, \ell))| = c(\pi(s, t)) \Leftrightarrow t$ lies on a shortest path from s to ℓ or s lies on a shortest path from ℓ to t



Landmark Hub Labeling (LHL)

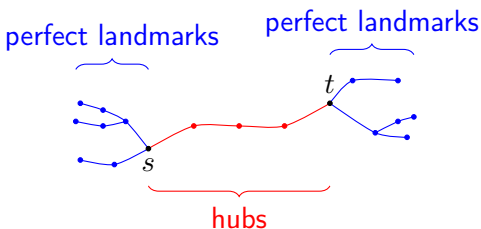
Landmark Hub Labeling (LHL) : [Storandt 2022]

- Generalisation of HL
- Allows also perfect landmark to cover a pair of nodes

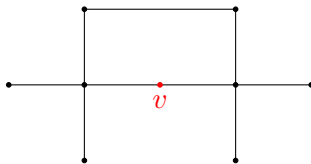
Definition

A node $\ell \in V$ is a perfect landmark for $s, t \in V$ if :

$|c(\pi(s, \ell)) - c(\pi(t, \ell))| = c(\pi(s, t)) \Leftrightarrow t$ lies on a shortest path from s to ℓ or s lies on a shortest path from ℓ to t



Efficiency of node coverage



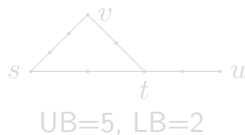
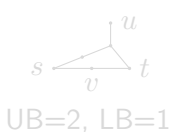
Node v is:

- a perfect landmark for 14 node pairs
- a hub for 15 node pairs
- a perfect landmark or a hub for 21 node pairs!

LHL : Query answering

Distance query answering :

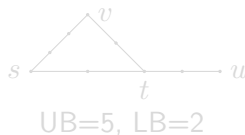
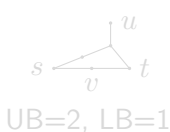
- Can be covered by either a hub or a perfect landmark
- Consider each $v \in L(s) \cap L(t)$ in both roles
- $UB := \min_{v \in L(s) \cap L(t)} (c(\pi(s, v)) + c(\pi(v, t)))$
- $LB := \max_{v \in L(s) \cap L(t)} |c(\pi(s, v)) - c(\pi(t, v))|$
- Check if LB tight : check if t is the node at distance LB from s on $\pi(s, v)$



LHL : Query answering

Distance query answering :

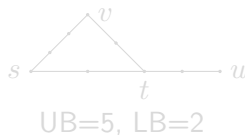
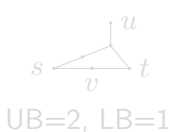
- Can be covered by either a hub or a perfect landmark
- Consider each $v \in L(s) \cap L(t)$ in both roles
- $UB := \min_{v \in L(s) \cap L(t)} (c(\pi(s, v)) + c(\pi(v, t)))$
- $LB := \max_{v \in L(s) \cap L(t)} |c(\pi(s, v)) - c(\pi(t, v))|$
- Check if LB tight : check if t is the node at distance LB from s on $\pi(s, v)$



LHL : Query answering

Distance query answering :

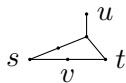
- Can be covered by either a hub or a perfect landmark
- Consider each $v \in L(s) \cap L(t)$ in both roles
- $UB := \min_{v \in L(s) \cap L(t)} (c(\pi(s, v)) + c(\pi(v, t)))$
- $LB := \max_{v \in L(s) \cap L(t)} |c(\pi(s, v)) - c(\pi(t, v))|$
- Check if LB tight : check if t is the node at distance LB from s on $\pi(s, v)$



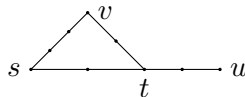
LHL : Query answering

Distance query answering :

- Can be covered by either a hub or a perfect landmark
- Consider each $v \in L(s) \cap L(t)$ in both roles
- $UB := \min_{v \in L(s) \cap L(t)} (c(\pi(s, v)) + c(\pi(v, t)))$
- $LB := \max_{v \in L(s) \cap L(t)} |c(\pi(s, v)) - c(\pi(t, v))|$
- Check if LB tight : check if t is the node at distance LB from s on $\pi(s, v)$



UB=2, LB=1

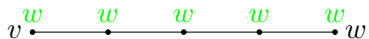


UB=5, LB=2

Specific kinds of labeling

Definition

A labeling is **path-consistent** (PC) if for each $w \in L(v)$, we also have $w \in L(u)$ for all u on the shortest path from v to w .



For PC-LHL, distance query in $\mathcal{O}(L_{max} + D)$. [Storandt 2022]

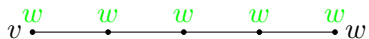
Definition

A labeling is **hierarchical** if there is a bijective ranking $r : V \rightarrow \{1, \dots, n\}$ such that for every $v \in V$, all $w \in L(v)$ are such that $r(w) \geq r(v)$.

Specific kinds of labeling

Definition

A labeling is **path-consistent** (PC) if for each $w \in L(v)$, we also have $w \in L(u)$ for all u on the shortest path from v to w .



For PC-LHL, distance query in $\mathcal{O}(L_{max} + D)$. [Storandt 2022]

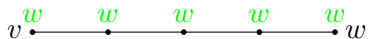
Definition

A labeling is **hierarchical** if there is a bijective ranking $r : V \rightarrow \{1, \dots, n\}$ such that for every $v \in V$, all $w \in L(v)$ are such that $r(w) \geq r(v)$.

Specific kinds of labeling

Definition

A labeling is **path-consistent** (PC) if for each $w \in L(v)$, we also have $w \in L(u)$ for all u on the shortest path from v to w .



For PC-LHL, distance query in $\mathcal{O}(L_{max} + D)$. [Storandt 2022]

Definition

A labeling is **hierarchical** if there is a bijective ranking $r : V \rightarrow \{1, \dots, n\}$ such that for every $v \in V$, all $w \in L(v)$ are such that $r(w) \geq r(v)$.

Canonical labelings

Canonical HHL

Given a node ranking r , the canonical HHL is such that, for $v, w \in V$, $w \in L(v)$ if and only if there is a shortest path starting at v on which w has the highest rank.

Canonical PC-HLHL

Given a node ranking r , the canonical PC-HLHL is such that, for $v, w \in V$, $w \in L(v)$ if and only if there is a maximal shortest path containing v and w on which w has the highest rank.

- Can be computed in **polytime**
- Smallest labeling size respecting r

Canonical labelings

Canonical HHL

Given a node ranking r , the canonical HHL is such that, for $v, w \in V$, $w \in L(v)$ if and only if there is a shortest path starting at v on which w has the highest rank.

Canonical PC-HLHL

Given a node ranking r , the canonical PC-HLHL is such that, for $v, w \in V$, $w \in L(v)$ if and only if there is a maximal shortest path containing v and w on which w has the highest rank.

- Can be computed in **polytime**
- **Smallest labeling size respecting r**

State of the art

HL : [Babenco et al. 2015]

- Minimising the max individual label set size or the total labeling size :
NP-hard
- $\mathcal{O}(\log n)$ **approximation** algorithm in $\mathcal{O}(n^3 \log n)$
- Best known approximation factor for HHL : $\mathcal{O}(\sqrt{n} \log n)$

HL on trees : [Angelidakis et al. 2015]

- HL = HHL
- NP-hardness?
- PTAS algorithm and 2-approximation algorithm

State of the art

HL : [Babenko et al. 2015]

- Minimising the max individual label set size or the total labeling size :
NP-hard
- $\mathcal{O}(\log n)$ **approximation** algorithm in $\mathcal{O}(n^3 \log n)$
- Best known approximation factor for HHL : $\mathcal{O}(\sqrt{n} \log n)$

HL on trees : [Angelidakis et al. 2015]

- HL = HHL
- NP-hardness?
- PTAS algorithm and 2-approximation algorithm

State of the art

Parametrized Upper Bounds for (H)HL :

- HHL upper bounded by $\mathcal{O}(t \log n)$ where t is the treewidth [Bauer et al. 2016]
- HHL upper bounded by $\mathcal{O}(h \log D)$ where h is the highway dimension [Abraham et al. 2010]
 - Intuitively, the highway dimension is small if, for every node, the set of lengthy shortest paths in its vicinity can be hit by a small number of nodes
 - Hard to compute but using a polynomial-time preprocessing, one can compute a HHL with maximum label size in $\mathcal{O}(h \log D \log n)$
- HL: expected upper bound (randomized preprocessing scheme) of $\mathcal{O}(\kappa \log n)$ with κ the skeleton dimension [Kosowski et al. 2017]
 - Intuitively, it measures how much shortest paths branch out in the middle section of lengthy paths from a given node

Too demanding and not used in practice

State of the art

Parametrized Upper Bounds for (H)HL :

- HHL upper bounded by $\mathcal{O}(t \log n)$ where t is the treewidth [Bauer et al. 2016]
- HHL upper bounded by $\mathcal{O}(h \log D)$ where h is the highway dimension [Abraham et al. 2010]
 - Intuitively, the highway dimension is small if, for every node, the set of lengthy shortest paths in its vicinity can be hit by a small number of nodes
 - Hard to compute but using a polynomial-time preprocessing, one can compute a HHL with maximum label size in $\mathcal{O}(h \log D \log n)$
- HL: expected upper bound (randomized preprocessing scheme) of $\mathcal{O}(\kappa \log n)$ with κ the skeleton dimension [Kosowski et al. 2017]
 - Intuitively, it measures how much shortest paths branch out in the middle section of lengthy paths from a given node

Too demanding and not used in practice

State of the art

Parametrized Upper Bounds for (H)HL :

- HHL upper bounded by $\mathcal{O}(t \log n)$ where t is the treewidth [Bauer et al. 2016]
- HHL upper bounded by $\mathcal{O}(h \log D)$ where h is the highway dimension [Abraham et al. 2010]
 - Intuitively, the highway dimension is small if, for every node, the set of lengthy shortest paths in its vicinity can be hit by a small number of nodes
 - Hard to compute but using a polynomial-time preprocessing, one can compute a HHL with maximum label size in $\mathcal{O}(h \log D \log n)$
- HL: expected upper bound (randomized preprocessing scheme) of $\mathcal{O}(\kappa \log n)$ with κ the skeleton dimension [Kosowski et al. 2017]
 - Intuitively, it measures how much shortest paths branch out in the middle section of lengthy paths from a given node

Too demanding and not used in practice

State of the art

Parametrized Upper Bounds for (H)HL :

- HHL upper bounded by $\mathcal{O}(t \log n)$ where t is the treewidth [Bauer et al. 2016]
- HHL upper bounded by $\mathcal{O}(h \log D)$ where h is the highway dimension [Abraham et al. 2010]
 - Intuitively, the highway dimension is small if, for every node, the set of lengthy shortest paths in its vicinity can be hit by a small number of nodes
 - Hard to compute but using a polynomial-time preprocessing, one can compute a HHL with maximum label size in $\mathcal{O}(h \log D \log n)$
- HL: expected upper bound (randomized preprocessing scheme) of $\mathcal{O}(\kappa \log n)$ with κ the skeleton dimension [Kosowski et al. 2017]
 - Intuitively, it measures how much shortest paths branch out in the middle section of lengthy paths from a given node

Too demanding and not used in practice

State of the art

LHL : [Storandt 2022]

- $\mathcal{O}(\log n)$ **approximation** algorithm in $\mathcal{O}(n^6)$ for PC-LHL
- Heuristic with a running time in $\mathcal{O}(n^3)$ for PC-LHL

Parametrized Upper Bounds for LHL : [Storandt 2022]

- Previous bounds also valid for (H)LHL
- PC-LHL upper bounded by $\mathcal{O}(gt)$, with gt the geodesic transversal number (smallest number of nodes needed to intersect every maximal shortest path)

State of the art

LHL : [Storandt 2022]

- $\mathcal{O}(\log n)$ **approximation** algorithm in $\mathcal{O}(n^6)$ for PC-LHL
- Heuristic with a running time in $\mathcal{O}(n^3)$ for PC-LHL

Parametrized Upper Bounds for LHL : [Storandt 2022]

- Previous bounds also valid for (H)LHL
- PC-LHL upper bounded by $\mathcal{O}(gt)$, with gt the geodesic transversal number (smallest number of nodes needed to intersect every maximal shortest path)

Plan

1 Introduction

2 Results

- Label size gaps
- Experimental label size comparison
- Faster query answering

3 Conclusion

Plan

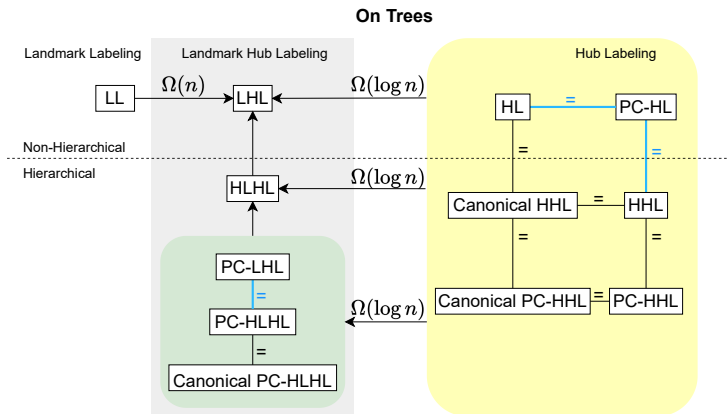
1 Introduction

2 Results

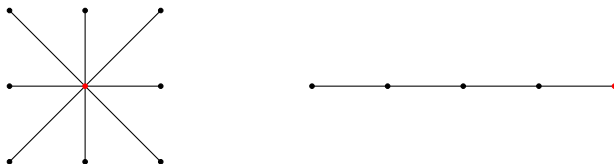
- Label size gaps
- Experimental label size comparison
- Faster query answering

3 Conclusion

Recap of the gaps on trees



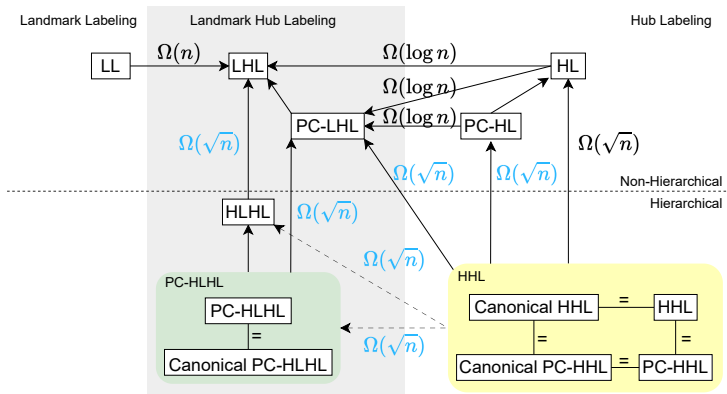
Some interesting known gaps



	star graph	path graph
HL	$\Theta(1)$	$\Theta(\log n)$
LL	$\Theta(n)$	$\Theta(1)$
LHL	$\Theta(1)$	$\Theta(1)$

Recap of the gaps on general graphs

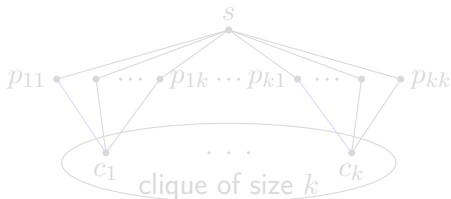
On General Graphs



New gap between HLHL and LHL

Theorem

There is a graph family for which the optimal HLHL size is $\Omega(\sqrt{n})$ times larger than the optimal label size of LHL.

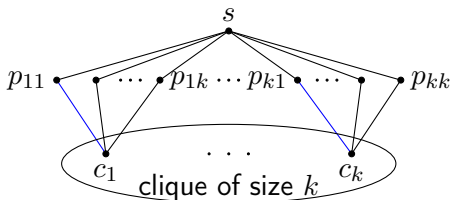


- LHL in $\mathcal{O}(n)$: $L(s) = \{s\}$,
 $L(p_{ix}) = \{s, c_i\}$,
 $L(c_i) = \{s, c_1, \dots, c_k\}$
- HLHL in $\Omega(n^{\frac{3}{2}})$:
 - $r(c_1) < \dots < r(c_k)$
 - Only max SP containing p_{ix} and c_j for $i < j$: (p_{ix}, c_i, c_j)
 - $c_i \notin L(c_j) \Rightarrow p_{ix} \in L(c_j)$ or $c_j \in L(p_{ix})$
 - $k \sum_{i=1}^{k-1} i = \frac{k^2(k-1)}{2} \in \Omega(n^{\frac{3}{2}})$

New gap between HLHL and LHL

Theorem

There is a graph family for which the optimal HLHL size is $\Omega(\sqrt{n})$ times larger than the optimal label size of LHL.

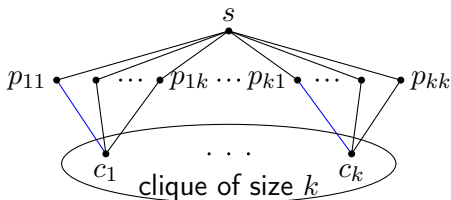


- LHL in $\mathcal{O}(n)$: $L(s) = \{s\}$,
 $L(p_{ix}) = \{s, c_i\}$,
 $L(c_i) = \{s, c_1, \dots, c_k\}$
- HLHL in $\Omega(n^{\frac{3}{2}})$:
 - $r(c_1) < \dots < r(c_k)$
 - Only max SP containing p_{ix} and c_j for $i < j$: (p_{ix}, c_i, c_j)
 - $c_i \notin L(c_j) \Rightarrow p_{ix} \in L(c_j)$ or $c_j \in L(p_{ix})$
 - $k \sum_{i=1}^{k-1} i = \frac{k^2(k-1)}{2} \in \Omega(n^{\frac{3}{2}})$

New gap between HLHL and LHL

Theorem

There is a graph family for which the optimal HLHL size is $\Omega(\sqrt{n})$ times larger than the optimal label size of LHL.

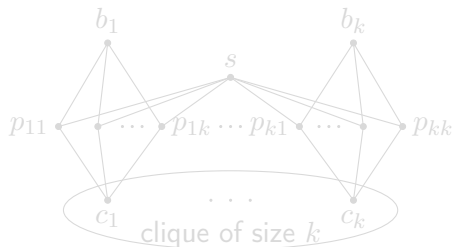


- LHL in $\mathcal{O}(n)$: $L(s) = \{s\}$,
 $L(p_{ix}) = \{s, c_i\}$,
 $L(c_i) = \{s, c_1, \dots, c_k\}$
- HLHL in $\Omega(n^{\frac{3}{2}})$:
 - $r(c_1) < \dots < r(c_k)$
 - Only max SP containing p_{ix} and c_j for $i < j$: (p_{ix}, c_i, c_j)
 - $c_i \notin L(c_j) \Rightarrow p_{ix} \in L(c_j)$ or $c_j \in L(p_{ix})$
 - $k \sum_{i=1}^{k-1} i = \frac{k^2(k-1)}{2} \in \Omega(n^{\frac{3}{2}})$

New gap between HHL and HLHL

Theorem

There is a graph family for which the optimal HHL size is $\Omega(\sqrt{n})$ times larger than the optimal label size of (PC-)HLHL.

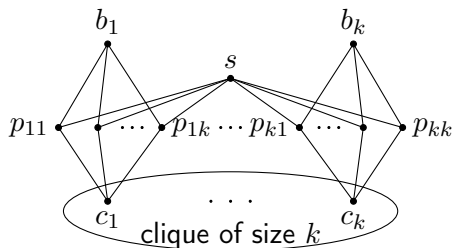


- PC-HLHL in $\mathcal{O}(n)$: rank from high to low: $s, b, c_k, \dots, c_1, p$,
 $L(s) = \{s\}$, $L(b_i) = \{s, b_i\}$,
 $L(p_{ix}) = \{s, b_i, p_{ix}\}$,
 $L(c_i) = \{s, c_i, \dots, c_k, b\}$
- HHL in $\Omega(n^{\frac{3}{2}})$:
 - $r(c_1) < \dots < r(c_k)$
 - For p_{ix} and c_j with $i < j$:
 $\pi(p_{ix}, c_j) = (p_{ix}, c_i, c_j)$
 - $c_i \notin L(c_j) \Rightarrow p_{ix} \in L(c_j)$ or
 $c_j \in L(p_{ix})$

New gap between HHL and HLHL

Theorem

There is a graph family for which the optimal HHL size is $\Omega(\sqrt{n})$ times larger than the optimal label size of (PC-)HLHL.

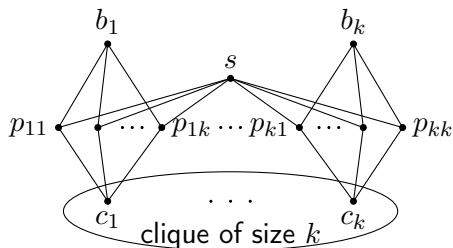


- PC-HLHL in $\mathcal{O}(n)$: rank from high to low: $s, b, c_k, \dots, c_1, p$,
 $L(s) = \{s\}$, $L(b_i) = \{s, b_i\}$,
 $L(p_{ix}) = \{s, b_i, p_{ix}\}$,
 $L(c_i) = \{s, c_i, \dots, c_k, b\}$
- HHL in $\Omega(n^{\frac{3}{2}})$:
 - $r(c_1) < \dots < r(c_k)$
 - For p_{ix} and c_j with $i < j$:
 $\pi(p_{ix}, c_j) = (p_{ix}, c_i, c_j)$
 - $c_i \notin L(c_j) \Rightarrow p_{ix} \in L(c_j)$ or
 $c_j \in L(p_{ix})$

New gap between HHL and HLHL

Theorem

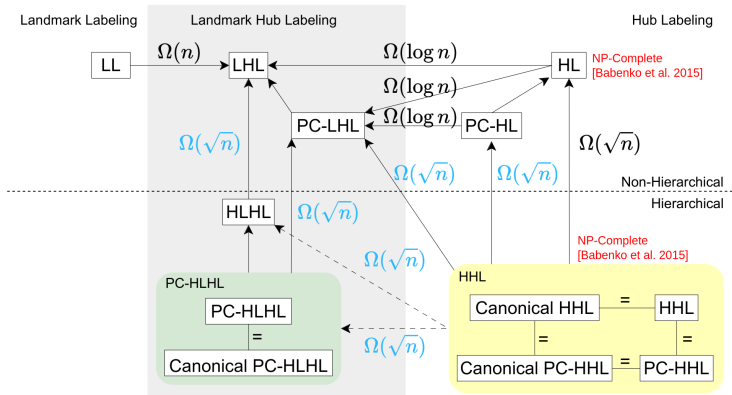
There is a graph family for which the optimal HHL size is $\Omega(\sqrt{n})$ times larger than the optimal label size of (PC-)HLHL.



- PC-HLHL in $\mathcal{O}(n)$: rank from high to low: $s, b, c_k, \dots, c_1, p$,
 $L(s) = \{s\}$, $L(b_i) = \{s, b_i\}$,
 $L(p_{ix}) = \{s, b_i, p_{ix}\}$,
 $L(c_i) = \{s, c_i, \dots, c_k, b\}$
- HHL in $\Omega(n^{\frac{3}{2}})$:
 - $r(c_1) < \dots < r(c_k)$
 - For p_{ix} and c_j with $i < j$:
 $\pi(p_{ix}, c_j) = (p_{ix}, c_i, c_j)$
 - $c_i \notin L(c_j) \Rightarrow p_{ix} \in L(c_j)$ or
 $c_j \in L(p_{ix})$

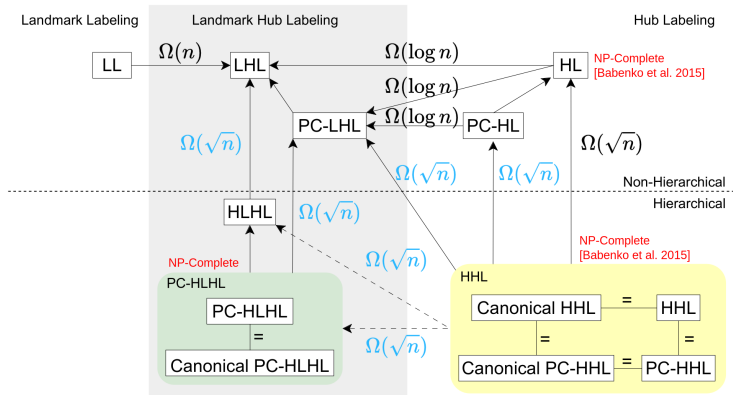
NP-Completeness

On General Graphs



NP-Completeness

On General Graphs



Plan

1 Introduction

2 Results

- Label size gaps
- Experimental label size comparison
- Faster query answering

3 Conclusion

Approximation algorithm: w-HL

Idea : Reformulate the cover property to get a Set Cover instance and use a greedy algorithm to compute a $\mathcal{O}(\log n)$ approximation [Cohen et al. 2003]

Set Cover

U - universe

$\mathcal{S}$ - collection of subsets S of U with weights $w(S)$

Goal: Find $\mathcal{S}' \subset \mathcal{S}$ of minimum weight such that $\cup_{S \in \mathcal{S}'} S = U$

- U : set of all (unordered) pairs of nodes (representing the set of all shortest path)
- $\mathcal{S}$: collection of all subsets of uncovered pairs that can be covered by the same node

Approximation algorithm: w-HL

Idea : Reformulate the cover property to get a Set Cover instance and use a greedy algorithm to compute a $\mathcal{O}(\log n)$ approximation [Cohen et al. 2003]

Set Cover

U - universe

$\mathcal{S}$ - collection of subsets S of U with weights $w(S)$

Goal: Find $\mathcal{S}' \subset \mathcal{S}$ of minimum weight such that $\cup_{S \in \mathcal{S}'} S = U$

- U : set of all (unordered) pairs of nodes (representing the set of all shortest path)
- $\mathcal{S}$: collection of all subsets of uncovered pairs that can be covered by the same node

Approximation algorithm: w-HL

Center graph of node v

In the center graph of node v , for all $u, w \in V$, there is an edge between node u and w if and only if they are not yet covered and $v \in \pi(u, w)$. The center graph does not contain isolated nodes.

w-HL :

- Find the maximum density subgraph of all center graphs
 - Densest subgraph problem: find subgraph G' with maximum density $d(G') = \frac{E'}{V'}$
 - Can be solved to optimality in polytime and there is a linear time 2-approx
- Add the center to the label set of the nodes in the maximum density subgraph
- Repeat until all pairs are covered

$\Rightarrow \mathcal{O}(\log n)$ approximation

Approximation algorithm: w-HL

Center graph of node v

In the center graph of node v , for all $u, w \in V$, there is an edge between node u and w if and only if they are not yet covered and $v \in \pi(u, w)$. The center graph does not contain isolated nodes.

w-HL :

- Find the maximum density subgraph of all center graphs
 - Densest subgraph problem: find subgraph G' with maximum density $d(G') = \frac{E'}{V'}$
 - Can be solved to optimality in polytime and there is a linear time 2-approx
- Add the center to the label set of the nodes in the maximum density subgraph
- Repeat until all pairs are covered

$\Rightarrow \mathcal{O}(\log n)$ approximation

Approximation algorithms

w-LHL :

- Adapt w-HL for LHL [Storandt 2022]
- For the center graph of node v , there is an edge between the uncovered pair (u, w) if and only if v lies on some shortest path containing u and w

$\Rightarrow \mathcal{O}(\log n)$ approximation

w-H(L)HL : [Babenko et al. 2015]

- Select the whole center graph of maximum density instead of its densest subgraph
- Since every center graph is chosen only once, the resulting labeling is hierarchical

Approximation algorithms

w-LHL :

- Adapt w-HL for LHL [Storandt 2022]
- For the center graph of node v , there is an edge between the uncovered pair (u, w) if and only if v lies on some shortest path containing u and w

⇒ $\mathcal{O}(\log n)$ approximation

w-H(L)HL : [Babenko et al. 2015]

- Select the whole center graph of maximum density instead of its densest subgraph
- Since every center graph is chosen only once, the resulting labeling is hierarchical

Approximation algorithms

g-H(L)HL : Select center graph which has the most edges

PC w-LHL :

- Resulting labeling from w-LHL not necessarily PC
- Given a valid LHL, one can produce a PC-LHL in polytime by running Dijkstra for each node v until it reaches all nodes w with $v \in L(w)$ and adding v in the label set of the nodes in the resulting shortest path tree

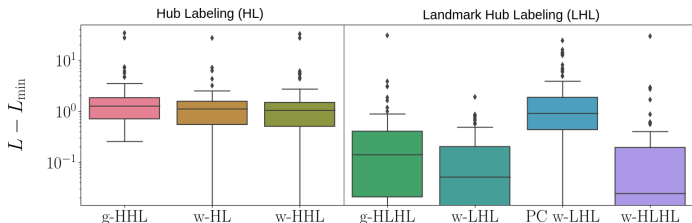
	g-HHL	w-HL	w-HHL	g-HLHL	w-LHL	PC w-LHL	w-HLHL
Appr. Factor	$\mathcal{O}(\sqrt{n} \log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\sqrt{n} \log n)$	-	$\mathcal{O}(\log n)$	-	-
Running time	$\mathcal{O}(n^3 \log n)$	$\mathcal{O}(n^3 \log n)$	$\mathcal{O}(n^3 \log n)$	$\mathcal{O}(n^3 \log n)$	$\mathcal{O}(n^4 \log n)$	$\mathcal{O}(n^4 \log n)$	$\mathcal{O}(n^3 \log n)$

Benchmark Sets

Benchmark Set	# Instances	$ V $	$ E $	$ E / V $
Open Street Map	12	[100, 2000]	[99, 2132]	≤ 2
PACE challenge 2020	100	[10, 491]	[15, 4100]	[1, 32]

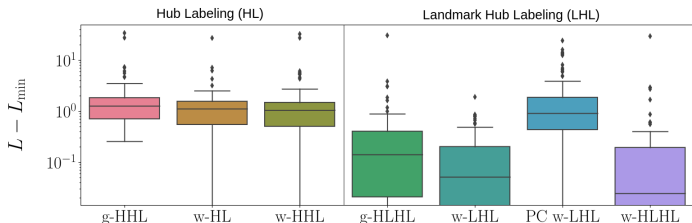
- Restrict our tests to road networks with up to 2000 nodes due to the large running time of the approximations
- Time-out set to 20 minutes per instance

Pace challenge 2020



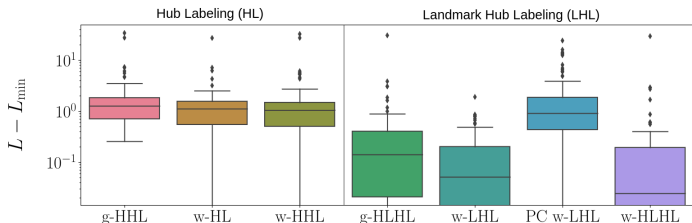
- In 95 out of 100 instances, the smallest label size is returned from one of the (H)LHL algorithms
- The ratio of HL and LHL size is up to 34 (on a graph with density of 15)

Pace challenge 2020



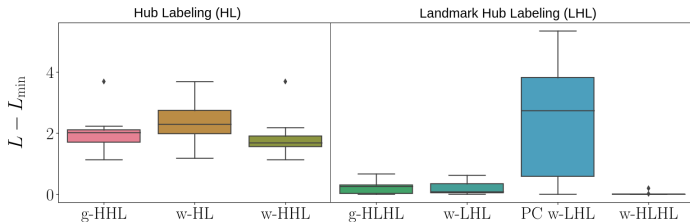
- In 95 out of 100 instances, the **smallest** label size is returned from one of the (H)LHL algorithms
- The ratio of HL and LHL size is up to 34 (on a graph with density of 15)

Pace challenge 2020



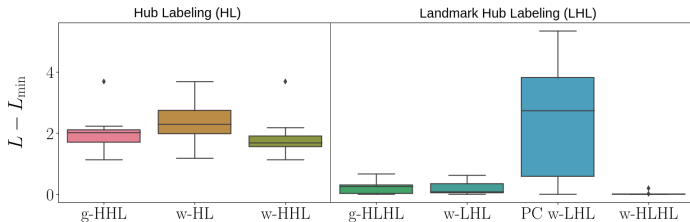
- In **95 out of 100** instances, the **smallest** label size is returned from one of the **(H)LHL** algorithms
- The ratio of HL and LHL size is up to **34** (on a graph with density of 15)

Open Street Map



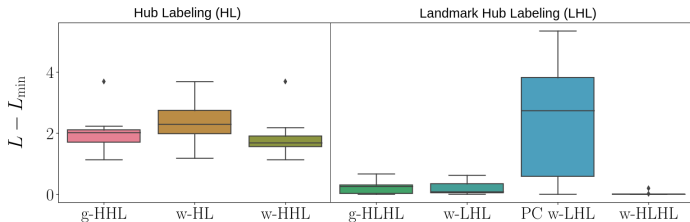
- On average, each node has at least 2 more labels in HLs than in LHLs
- The ratio of HL to LHL size is up to 3

Open Street Map



- On average, each node has at least **2 more labels in HLs** than in LHLs
- The ratio of HL to LHL size is up to **3**

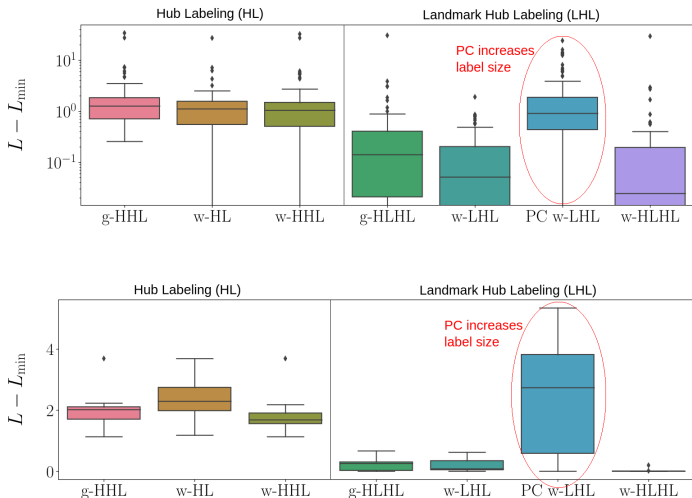
Open Street Map



- On average, each node has at least **2 more labels in HLs** than in LHLs
- The ratio of HL to LHL size is up to **3**

Path consistency property

- PC w-LHL smaller than w-HL in 51 out of 100 instances of PACE



Path consistency property

Recall :

- Non-PC Landmark Hub Labelings are better than Hub Labelings
- PC was designed for fast calculating shortest path distance instead of LB and UB

PC limitations :

- Large label size
- Distance answering requires path backtracking
 - PC-LHL: $\mathcal{O}(L_{max} + D)$
 - HL: $\mathcal{O}(L_{max})$

How to answer the shortest path distance query efficiently, based on LHL?

Path consistency property

Recall :

- Non-PC Landmark Hub Labelings are better than Hub Labelings
- PC was designed for fast calculating shortest path distance instead of LB and UB

PC limitations :

- Large label size
- Distance answering requires path backtracking
 - PC-LHL: $\mathcal{O}(L_{max} + D)$
 - HL: $\mathcal{O}(L_{max})$

How to answer the shortest path distance query efficiently, based on LHL?

Plan

1 Introduction

2 Results

- Label size gaps
- Experimental label size comparison
- Faster query answering

3 Conclusion

The LCA trick

Goal : A more efficient way to characterize whether a node v is a perfect landmark for a pair s, t

Lowest Common Ancestor :

- T_v : shortest path tree emerging from v
- $LCA_v(s, t)$: lowest common ancestor of s and t in T_v
- For a given tree T_v , an LCA data structure can be computed in linear time and space, which allows to answer $LCA_v(s, t)$ queries in $\mathcal{O}(1)$ [Bender et al. 2005]

Theorem

The node v is a perfect landmark for $s, t \in V$ if and only if $LCA_v(s, t) = s$ or $LCA_v(s, t) = t$.

Theorem

The node v is a hub for $s, t \in V$ only if $LCA_v(s, t) = v$.

The LCA trick

Goal : A more efficient way to characterize whether a node v is a perfect landmark for a pair s, t

Lowest Common Ancestor :

- T_v : shortest path tree emerging from v
- $LCA_v(s, t)$: lowest common ancestor of s and t in T_v
- For a given tree T_v , an LCA data structure can be computed in linear time and space, which allows to answer $LCA_v(s, t)$ queries in $\mathcal{O}(1)$ [Bender et al. 2005]

Theorem

The node v is a perfect landmark for $s, t \in V$ if and only if $LCA_v(s, t) = s$ or $LCA_v(s, t) = t$.

Theorem

The node v is a hub for $s, t \in V$ only if $LCA_v(s, t) = v$.

The LCA trick

Goal : A more efficient way to characterize whether a node v is a perfect landmark for a pair s, t

Lowest Common Ancestor :

- T_v : shortest path tree emerging from v
- $LCA_v(s, t)$: lowest common ancestor of s and t in T_v
- For a given tree T_v , an LCA data structure can be computed in linear time and space, which allows to answer $LCA_v(s, t)$ queries in $\mathcal{O}(1)$ [Bender et al. 2005]

Theorem

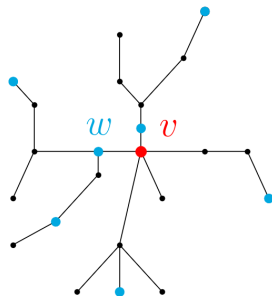
The node v is a perfect landmark for $s, t \in V$ if and only if $LCA_v(s, t) = s$ or $LCA_v(s, t) = t$.

Theorem

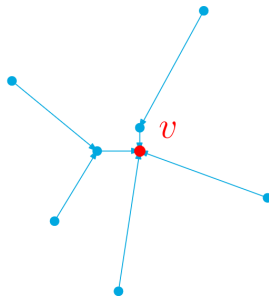
The node v is a hub for $s, t \in V$ only if $LCA_v(s, t) = v$.

Compressed tree

$v \in L(w)$



shortest path tree T_v

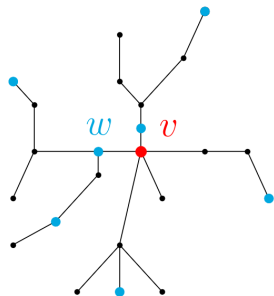


compressed tree T_v^*

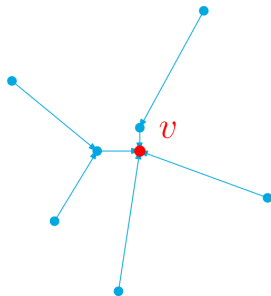
$$\Rightarrow \sum_{v \in V} |L(v)| = \sum_{v \in V} |T_v^*|$$

Compressed tree

$v \in L(w)$



shortest path tree T_v

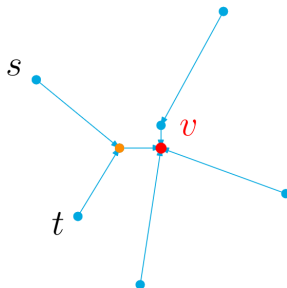


compressed tree T_v^*

$$\Rightarrow \sum_{v \in V} |L(v)| = \sum_{v \in V} |T_v^*|$$

Distance query answering

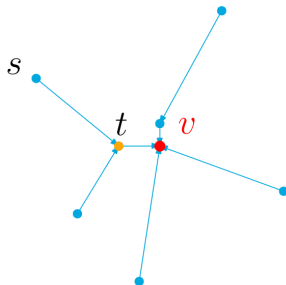
- Find common node IDs in $L(s)$ and $L(t)$ in linear time
- For each such node v , query the LCA_v data structure



compressed tree T_v^*

$LCA_v(s, t) \notin \{s, t, v\} \Rightarrow v$ is not a hub or a perfect landmark

Distance query answering

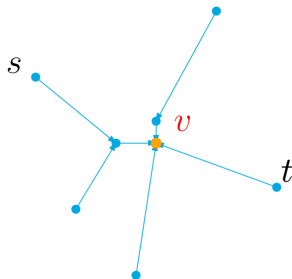


compressed tree T_v^*

$LCA_v(s, t) \in \{s, t\} \Rightarrow v$ is a perfect landmark

$\Rightarrow$ Return $|c(\pi(s, v)) - c(\pi(t, v))|$

Distance query answering



compressed tree T_v^*

$LCA_v(s, t) = v \Rightarrow v$ could be a hub

Update UB (initially set to $+\infty$) if $c(\pi(s, v)) + c(\pi(v, t)) < UB$

Distance query answering

At the end, if no perfect landmark was detected, return UB which is now certified to be correct

⇒ Distance queries can be answered with LHL in $\mathcal{O}(L_{max})$

Goodbye PC

Distance query answering

At the end, if no perfect landmark was detected, return UB which is now certified to be correct

⇒ Distance queries can be answered with LHL in $\mathcal{O}(L_{max})$

Goodbye PC

Distance query answering

At the end, if no perfect landmark was detected, return UB which is now certified to be correct

⇒ Distance queries can be answered with LHL in $\mathcal{O}(L_{max})$

Goodbye PC

Plan

1 Introduction

2 Results

- Label size gaps
- Experimental label size comparison
- Faster query answering

3 Conclusion

Summary and future work

Summary :

- Theoretically and experimentally show that LHL can perform much better than HL
- Provide hardness result
- Improve query routines for Landmark Hub Labeling

Future work :

- How to construct LHL efficiently? Good heuristic for LHL, s.t. it can scale to large graphs?
- Assign node IDs in a LHL s.t. perfect landmarks are identified early for many node pairs?
 - LCA-based query routine allows for stopping early if a perfect landmark is identified
 - Could decrease the query time in practice

Summary and future work

Summary :

- Theoretically and experimentally show that LHL can perform much better than HL
- Provide hardness result
- Improve query routines for Landmark Hub Labeling

Future work :

- How to construct LHL efficiently? Good heuristic for LHL, s.t. it can scale to large graphs?
- Assign node IDs in a LHL s.t. perfect landmarks are identified early for many node pairs?
 - LCA-based query routine allows for stopping early if a perfect landmark is identified
 - Could decrease the query time in practice

Questions

Thank you for listening :)

Questions?

1 Introduction

2 Results

- Label size gaps
- Experimental label size comparison
- Faster query answering

3 Conclusion

Unique shortest path assumption

- $\tilde{w}(e) = (w(e), w'(e))$
- Path length: sum on each component, comparison: lexicographic order
- w' such that unique shortest path with w' :
 - $w'(e)$ chosen independently and uniformly at random from $\{1, 2, \dots, n^4\} \Rightarrow$ all shortest path unique with prob. $1 - \mathcal{O}(1/n)$ by isolation lemma
 - Index the edges from 1 to m and define $w'(e_i) = 2^i$

Path queries for HL

- Not only the distances to the hub nodes are stored with the labels but also the first edge on the shortest path
- First issue a distance query to identify the hub w and $c(s, t)$
- $\pi(s, t)$ is the concatenation of $\pi(s, w)$ and $\pi(w, t)$. Extract the two subpaths independently:
 - To extract any $a - b$ path: identify w_{ab} with a distance query
 - If $w_{ab} \neq a$, then the first edge on $\pi(a, b)$ is the one associated to $w_{ab} \in L(a)$
 - Otherwise, $a \in L(b)$ and the edge associated to this label is the last edge on $\pi(a, b)$
 - Thus we get the first or last edge of the path. Recursively repeat the procedure

$\Rightarrow \mathcal{O}(D \times L_{max})$

Path queries with PC

- To each node $l \in L(s)$, we assign a pointer to the entry of $l \in L(s')$ where s' is the node after s on $\pi(s, l)$ (space consumption only increases linearly)
- Shortest path from s to $l \in L(s)$ can be retrieved by following the pointer towards l iteratively
- Path query: Distance query + path retrieval $\Rightarrow \mathcal{O}(L_{max} + D)$

Detail on the data structure for the new query routine

- Conventionally, each node simply stores its labels as a list of nodes with corresponding distance values (sorted by node ID to enable efficient computation of label intersections)
- Each node $v \in V$ stores a partial shortest path tree T_v^* defined as the smallest subtree of T_v that contains all nodes w with $v \in L(w)$
- Shortest path distances are stored along with the tree nodes as well as a pointer to the root node. For each T_v^* , we compute an LCA data structure
- The nodes additionally hold a list of pointers to their positions in these partial shortest path trees, sorted by the ID of the respective root node

Canonical PC-HLHL

Given a node ranking r , the canonical PC-HLHL is such that, for $v, w \in V$, $w \in L(v)$ if and only if there is a maximal shortest path containing v and w on which w has the highest rank.

- It is a valid PC-HLHL
- We have, for all $v \in V$ $L_c(v) \subset L(v)$ for L any valid PC-HLHL with same ranking r :
 - $w \in L_c(v)$ and assume $w \notin L(v)$
 - There is a maximal shortest path π containing v and w on which w has the highest rank. Let s and t be its end nodes. Based on the unique shortest path assumption, this is the only geodesic that contains both s and t
 - s and t must have a common node from π in their label. By PC, all other nodes on π must have the same node in their label
 - If this is not w , we have $w' \in L(w)$ with $r(w') < r(w)$: contradiction

2-approximation on tree

Definition

Consider a tree T on n vertices. We say that a vertex u is a balanced separator vertex if every connected component of $T \setminus u$ has at most $n/2$ vertices.

Theorem

Every tree has a balanced separator vertex. (can be found in linear time)

- Find a balanced separator vertex r in T
- Remove r and recursively find a HL in each subtree T_i of $T \setminus r$. Let H' be the labeling obtained by the recursive procedure
- Return $H(u) = H'(u) \cup \{r\}$ and $H(r) = \{r\}$

$\Rightarrow \mathcal{O}(n \log n)$ time

Use a primal-dual approach to prove the approx: define a dual feasible solution whose cost is at least half of the sol. of the algo.

Primal LP

$$\min \sum_{u \in V} \sum_{v \in V} x_{uv}$$

such that:

$$\sum_{w \in P_{uv}} y_{uvw} \geq 1, \forall \{u, v\} \in I$$

$$x_{uw} \geq y_{uvw}, \forall \{u, v\} \in I, \forall w \in P_{uv}$$

$$x_{vw} \geq y_{uvw}, \forall \{u, v\} \in I, \forall w \in P_{uv}$$

$$x_{uv} \geq 0, \forall \{u, v\} \in V \times V$$

$$y_{uvw} \geq 0, \forall \{u, v\} \in I, \forall w \in P_{uv}$$

Here, I is the set of all unordered pairs of vertices, including pairs (u, u) and x_{uv} represents whether $v \in L(u)$ or not

$\mathcal{O}(\log n)$ approx. for PC-LHL

- Greedy set cover does not necessarily return a PC labeling
- $\mathcal{S}$ restricted to sets $S(w, W)$ where W is a connected subgraph of the shortest path tree emerging from w (such sets W are called trimmed shortest path tree rooted at w)
- Find a trimmed shortest path tree with close to maximum density

Heuristic for PC-LHL

- Extract all maximal geodesics and compute a greedy hitting set
- Find a gt -set of size $\mathcal{O}(gt \log n)$ in $\mathcal{O}(n^3)$

PC-LHL=PC-HLHL on trees

Theorem

For any tree $T = (V, E)$ and any valid PC-LHL L on T , there exists a hierarchical PC-LHL L' such that $|L'| \leq |L|$.

- T_u : u root of T_u . There is a path $\pi(u, v)$ in T_u if $v \in L(u)$
- Let $w \in L(u) \cap L(v)$, then $w \in T_u$ and $w \in T_v$. Helly Property: given a family of subtrees of T , if every two subtrees intersect, then all subtrees intersect on at least one node r
- First, $L' = L$. Then r highest rank and add r in every label set. Remove $w \in L'(u)$ such that $r \in \pi(u, w)$. L' is still a valid PC-LHL with $|L'| \leq |L|$

PC-LHL=PC-HLHL on trees

Theorem

For any tree $T = (V, E)$ and any valid PC-LHL L on T , there exists a hierarchical PC-LHL L' such that $|L'| \leq |L|$.

- Now deal with $T_1, \dots, T_k$, the subtrees rooted at the neighbors of r , separately. Consider a T_i and the degree two path starting from its root and ending at a node w . Remove from this path the nodes u in the labels that are not r . L' is still a valid PC-LHL and $|L'| \leq |L|$
- T' the tree obtained by removing the degree two path. $l_1, \dots, l_p$ the leaves of T' . $r \notin \pi(l_i, l_j)$. There exists $u \in L(l_i) \cap L(l_j)$, and $u \in L(w)$ for all $w \in \pi(l_i, l_j)$.
- As a pair of nodes always lies on a path from a leaf to another, for any $u, v \in T'$, $L(u) \cap L(v) \neq \emptyset$. Thus T'_u and T'_v have an intersection and we can apply again the Helly property...

NP-Completeness of PC-HLHL

Reduction

$G = (V, E)$ graph with degree of at least 3. Let $G^+ = (V^+, E^+)$ be the graph obtained by connecting one single node to each node in G . Denote $v' \in V^+ \setminus V$ as the node added to the node $v \in V$, called leaf of v .

Theorem

The graph G has an HHL of size k if and only if G^+ has a PC-HLHL of size $2k$.

- Assume G has a canonical HHL L of size k with the rank r
- Assign each non-leaf node v in G^+ with the rank of $r(v) + n$ and assign the rank of leaves randomly from 1 to n . $L(v') = L(v)$
- Since all maximal geodesics with end v' contain v , and all shortest paths reaching v are covered in the HHL, then all maximal geodesics are also covered
- The constructed labeling is a canonical PC-HLHL with a size of $2k$

NP-Completeness of PC-HLHL

Reduction

$G = (V, E)$ graph with degree of at least 3. Let $G^+ = (V^+, E^+)$ be the graph obtained by connecting one single node to each node in G . Denote $v' \in V^+ \setminus V$ as the node added to the node $v \in V$, called leaf of v .

Theorem

The graph G has an HHL of size k if and only if G^+ has a PC-HLHL of size $2k$.

- L an optimal PC-HLHL of G^+ of size $2k$
- There is an optimal PC-HLHL L such that all leaves have lower ranks than the nodes in V
- $L(G^+[G])$ is an HHL for nodes in V with $|L(G^+[G])| = k$
- If $L(G^+[G])$ is not an optimal HHL, then we can construct a PC-HLHL for G^+ with a smaller HHL

Improved approximation algorithms

Checking if a pair s, t is covered by a node v :

- Instead of storing all maximal geodesics, use a distance matrix to store only the shortest path distances
- Reduces the space consumption from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$
- Given a node v and a pair (s, t) , v is a hub or a perfect landmark if and only if $c(s, v) + c(v, t) = c(s, t)$ or $c(s, t) + c(v, t) = c(s, v)$ or $c(s, v) + c(s, t) = c(v, t) \Rightarrow$ **constant time**

Avoid unnecessary recomputation of the MDS of a node's center graph:

- The MDS of a graph cannot be increased by removing nodes or edges in the graph
- Max-heap to store for each node the density of its MDS
- In each iteration, a threshold d is initially set to $-\infty$. While the node v at the top has a corresponding stored density of at least d , we recompute the density md of its new center graph and update $d = \max(d, md)$. Then, the node is pushed back to the heap

Approximation factor HHL

Lemma

If we run the greedy set cover algorithm where in each iteration we pick a set whose coverage to cost ratio is at least $1/f(n)$ fraction of the maximum coverage to cost ratio, then we get a cover of cost within an $\mathcal{O}(f(n) \log n)$ factor of optimal.

- If G is a graph with no isolated nodes, then G is an $\mathcal{O}(\sqrt{n})$ -approximation of the maximum density subgraph of $G \Rightarrow$ w-HHL is a $\mathcal{O}(\sqrt{n} \log n)$ approximation
- For g-HHL, the density of the chosen center graph is a $\sqrt{2n}$ -approximation of the maximum density of any subgraph $\Rightarrow$ g-HHL is a $\mathcal{O}(\sqrt{n} \log n)$ approximation

Experimental results

	g-HHL		w-HL		w-HHL		g-HLHL		w-LHL			w-HLHL	
	$ L $	t(s)	$ L $	t(s)	$ L $	t(s)	$ L $	t(s)	$ L $	t(s)	PC	$ L $	t(s)
1	5.7	≤ 1	5.7	≤ 1	5.7	≤ 1	2.0	≤ 1	2.0	1	0	2.0	≤ 1
2	4.7	≤ 1	4.8	≤ 1	4.7	≤ 1	3.6	≤ 1	3.6	≤ 1	0.3	3.5	≤ 1
3	5.3	≤ 1	5.5	≤ 1	5.4	≤ 1	4.0	≤ 1	4.0	1	0.7	4.4	≤ 1
4	5.2	≤ 1	5.1	≤ 1	5.1	≤ 1	2.9	≤ 1	3.0	1	0.3	3.0	≤ 1
5	9.5	72	9.9	136	9.4	68	8.1	84	8.5	285	4.7	7.9	82
6	10.3	74	10.4	148	10.2	69	8.5	84	8.9	315	4.2	8.7	84
7	9.4	73	9.5	146	9.3	68	7.7	82	7.5	311	2.8	7.4	83
8	8.9	74	9.2	161	8.7	71	7.1	87	7.1	363	2.4	6.9	88
9	12.3	484	13.6	848	12.0	437	10.8	557				10.5	523
10	12.4	494	13.2	899	12.3	445	11.0	587				10.3	532
11	11.4	475	11.9	856	11.0	438	9.7	541				9.6	530
12	10.0	489	10.4	1019	10.0	461	8.1	584				7.8	572

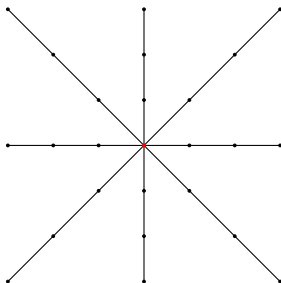
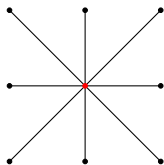
Theorem

v is a perfect landmark for $s, t \in V$ if and only if $LCA_v(s, t) = s$ or t . v is a hub for $s, t \in V$ if and only if $LCA_s(t, v) = v$.

- 1 Preprocessing phase: Compute T_v , the shortest path tree rooted at v , for all $v \in V$. For each T_v , apply a preprocessing phase for LCA_v which will allow future LCA_v queries to be answered in constant time.
- 2 Verification phase: Given a labeling L , verify that each pair of nodes $s, t \in V$ is covered by either a hub or a perfect landmark. Let $v \in L(s) \cap L(t)$. If $LCA_v(s, t) = s$ or t or $LCA_s(t, v) = v$, look at the next pair of nodes. Otherwise, if no node in $L(s) \cap L(t)$ satisfies this, L is not a LHL.

$\Rightarrow$ **space:** $\mathcal{O}(n^2)$, **preprocessing time:** $\mathcal{O}(nm + n^2 \log(n))$,
verification: $\mathcal{O}(n^2 \times L_{max})$

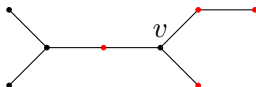
DPC: idea of the algorithm



The DPC heuristic

DPC heuristic :

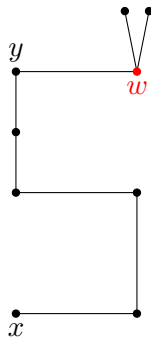
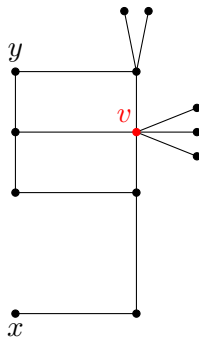
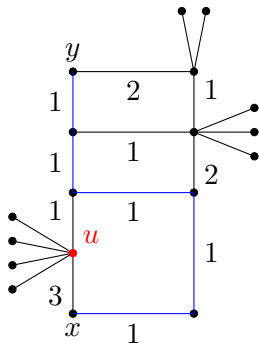
- Select v of highest cost $c(v) = \max(deg(v), path(v))$
- Add v to the label of all nodes in the connected component
- Remove from the tree the nodes contributing to $path(v)$ and v
- Apply this until there is no node left



Theorem

The DPC heuristic returns a correct PC-HLHL.

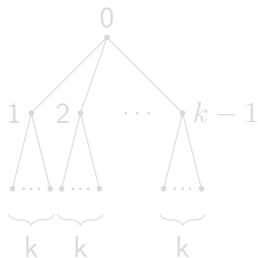
DPC on general graphs



DPC: some bad cases

Theorem

There are trees for which optimal PC-HLHL total label size is in $\Theta(n)$ and the DPC heuristic yields a labeling of total label size in $\Theta(n\sqrt{n})$.

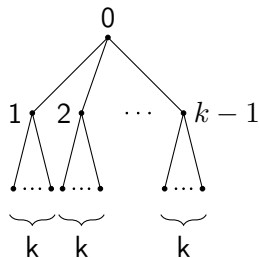


- PC-HLHL in $\Theta(n)$: every node contains its ancestors and itself
- DPC heuristic in $\Theta(n\sqrt{n})$:
 - Select intermediate nodes
 - $L(0) = \{0, 1, \dots, k-1\}$
 - $L(i) = \{1, \dots, i\}$ for $1 \leq i \leq k-1$
 - $L(u) = \{1, \dots, i, u\}$ for u a leaf in the subtree rooted at i
 - $k + \sum_{i=1}^{k-1} i + \sum_{i=1}^{k-1} k(i+1) = \Theta(n\sqrt{n})$

DPC: some bad cases

Theorem

There are trees for which optimal PC-HLHL total label size is in $\Theta(n)$ and the DPC heuristic yields a labeling of total label size in $\Theta(n\sqrt{n})$.

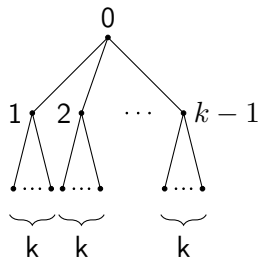


- PC-HLHL in $\Theta(n)$: every node contains its ancestors and itself
- DPC heuristic in $\Theta(n\sqrt{n})$:
 - Select intermediate nodes
 - $L(0) = \{0, 1, \dots, k-1\}$
 - $L(i) = \{1, \dots, i\}$ for $1 \leq i \leq k-1$
 - $L(u) = \{1, \dots, i, u\}$ for u a leaf in the subtree rooted at i
 - $k + \sum_{i=1}^{k-1} i + \sum_{i=1}^{k-1} k(i+1) = \Theta(n\sqrt{n})$

DPC: some bad cases

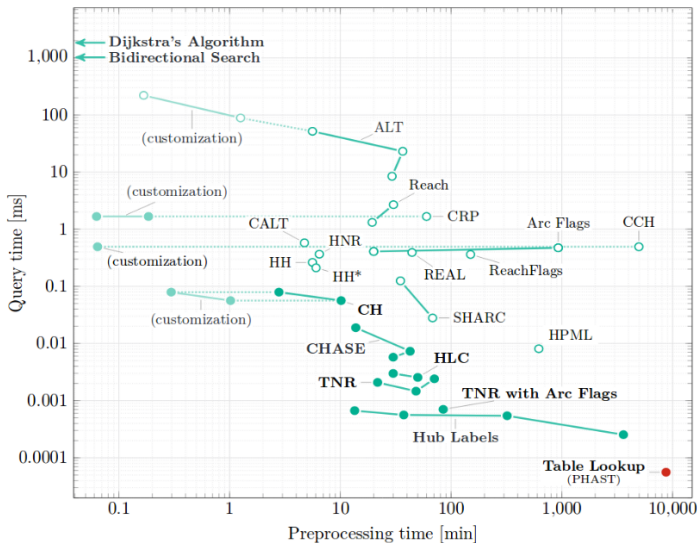
Theorem

There are trees for which optimal PC-HLHL total label size is in $\Theta(n)$ and the DPC heuristic yields a labeling of total label size in $\Theta(n\sqrt{n})$.



- PC-HLHL in $\Theta(n)$: every node contains its ancestors and itself
- DPC heuristic in $\Theta(n\sqrt{n})$:
 - Select intermediate nodes
 - $L(0) = \{0, 1, \dots, k-1\}$
 - $L(i) = \{1, \dots, i\}$ for $1 \leq i \leq k-1$
 - $L(u) = \{1, \dots, i, u\}$ for u a leaf in the subtree rooted at i
 - $k + \sum_{i=1}^{k-1} i + \sum_{i=1}^{k-1} k(i+1) = \Theta(n\sqrt{n})$

Preprocessing-Based Techniques



ALT (A^* +Landmarks+Triangle inequality) [2005]

Idea : To get an admissible heuristic for A^* , precompute paths costs from all nodes from/to some selected nodes ('landmarks')

Heuristic : With target t , $h(v) = \max_{l \in L} |c(\pi(v, l)) - c(\pi(v, t))|$

- Typically, a small number of landmarks suffices to have good bounds
- In practice A^* might even be slower than Dijkstra on road networks, but ALT is faster by a factor ≥ 10

Arc-Flags [2009]

Idea : Leverage the fact that shortest path trees of close nodes often overlap significantly to store those in a compact fashion

Preprocessing :

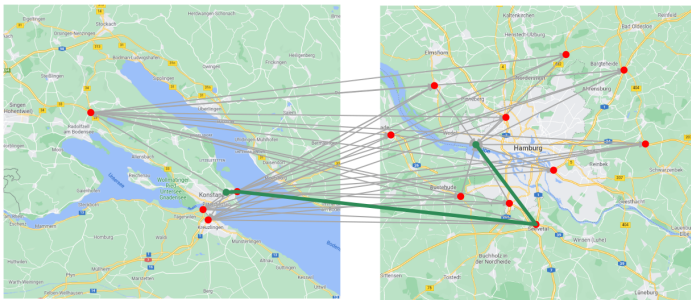
- Partition the network into k parts
- For each part, identify its border nodes
- Compute the reverse shortest path tree from each border node
- Assign to each edge a bit-vector of length k and set the flag for part i to True iff the edge is on a SP to some border node of part i or within part i

Query answering : Run Dijkstra and only relax edges that are flagged True for the part of the target

Faster than Dijkstra by a factor of 100-1000 in practice

Transit Nodes [2009]

Idea : There is a small set of locations out of which one needs to be passed to go 'far away' on a shortest path



If all distances $\text{green} \rightarrow \text{red}$ and $\text{red} \rightarrow \text{red}$ and $\text{red} \rightarrow \text{green}$ were known, one could easily determine the shortest path distance (without Dijkstra).

Skeleton Dimension

GEOMETRIC REALIZATION

Edges are subdivided into infinitely many infinitely short edges. Weights are interpolated linearly.

NODE REACH

The reach of a node $nr(v)$ is the longest path length from v towards a leaf in its subtree.

TREE SKELETON

$$T^* := \{v \in T \mid nr(v) \geq 1/2 \cdot c(s, v)\}$$

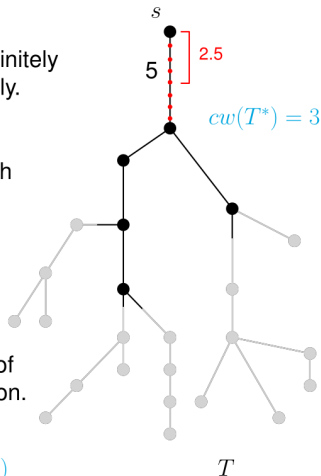
CUT WIDTH

For given $r > 0$ the cut width is the number of nodes at distance r from s in the tree skeleton.

$$cw(T^*) = \max_{r>0} |Cut_r(T^*)|$$

SKELETON DIMENSION

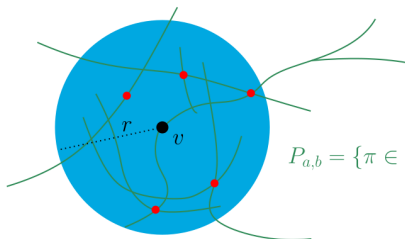
$$\kappa := \max_{s \in V} cw(T_s^*)$$



Highway Dimension

INTUITION

The HD is small if, for all nodes, the set of 'long' shortest paths in its 'vicinity' can be hit by few nodes.



$$P_{a,b} = \{\pi \in G \mid c(\pi) \in (a, b]\}$$



Ball of radius r $B_r(v) = \{w \in V \mid c(v, w) \leq r\}$

HD2 Smallest integer $hd2$ such that $\forall r > 0$ and $\forall v \in V$, there is a HS H with $|H| \leq hd2$ for paths in $P_{r,2r}$ that intersect $B_{2r}(v)$.